

TEL102 - Control 4

Semestre 2, 2025

Fecha: 18/11/2025, **Duración:** 1h 10min (30 min Parte A, 40 min Parte B)

Instrucciones generales

- **Parte A:** trabajo en papel, sin computador. Se analiza y diseña una clase Qt en C++ (sin interfaz), usando **signals y slots**.
- **Parte B:** implementación en Qt reutilizando lo diseñado en la Parte A.

Se permite usar **IA** en la Parte B, pero debe registrarse en [Readme.md](#):

- Datos del estudiante.
- Prompts usados.
- Modelo y versión.
- Correcciones hechas al código sugerido por la IA.
- Breve reflexión de uso.

Se entregan archivos base del proyecto en Aula USM. Solo se pueden usar elementos **vistos en clases**.

Contexto: SpaceDock Oxygen Monitor

Eres responsable del sistema de soporte vital de la estación espacial **SpaceDock**. La seguridad de la tripulación depende de mantener estable el **nivel de oxígeno** en el módulo principal.

El equipo desea construir una aplicación Qt que permita:

1. **Ajustar el nivel de oxígeno del módulo** usando dos representaciones:
 - **Porcentaje de oxígeno (%)** valor principal usado por el sistema.
 - **Presión parcial de oxígeno en kPa** valor derivado y sincronizado.
2. **Registrar las 12 últimas mediciones de oxígeno (%)**.
3. Mostrar dichas mediciones como un **gráfico de línea**.

La aplicación se organiza en tres componentes:

- **OxygenController** Clase entregada. Ajusta y sincroniza % y kPa. Emite signals cuando cambian.
- **OxygenHistory** Clase que el estudiante debe diseñar y completar. Almacena últimos 12 valores.
- **MainWindow** Interfaz del panel SpaceDock. Conecta diales, displays y gráfico usando signals/slots.

Todo el código base se entrega en AULA USM.

Parte A - Diseño de **OxygenHistory** (en papel) (40 puntos)

En esta parte, usted trabajará **solo en papel**.

Debe completar el diseño de la clase **OxygenHistory** a partir del siguiente esqueleto mínimo:

oxygenhistory.h (base entregada)

```
#ifndef OXYGENHISTORY_H
#define OXYGENHISTORY_H

#include <QObject>
#include <vector>

// Historial de niveles de oxígeno en porcentaje (%)

class OxygenHistory : public QObject {
    Q_OBJECT

public:
    OxygenHistory(QObject *parent = nullptr);

    // COMPLETAR

private:
    std::vector<float> m_values; // historial en %
    static const int MAX_VALUES = 12;
};

#endif // OXYGENHISTORY_H
```

A.1 Requerimientos obligatorios del diseño

Debe definir e implementar en papel:

1. Métodos (obligatorio) (20 pts)

- Agregar una medición de oxígeno (con máximo de **12 elementos**).
- Obtener la cantidad de mediciones.
- Acceder a un valor específico (**valueAt(i)**).

2. Signals (10 pts)

Ejemplos esperados:

- Notificar que el historial cambió (para actualizar el gráfico).
- (Opcional) Notificar que se agregó una nueva medición.

3. Slots (10 pts)

- Slot que reciba oxígeno en porcentaje (`int`), emitido desde `OxygenController`.

Debe permitir la conexión:

```
QObject::connect(&controller,  
                &OxygenController::oxygenPercentChanged,  
                &history,  
                /* su slot */);
```

A.2 Consideraciones adicionales

En su diseño en papel debe dejar claro:

- Cómo se mantiene el máximo de 12 valores.
- Qué tipo retorna cada método.
- Qué signal(s) se emitirán cuando cambie el historial.
- Cómo manejar índices inválidos.

No necesita escribir código completo de Qt, pero sí un diseño claro y evaluable.

Parte B - Implementación de SpaceDock Oxygen Monitor (con computador) (60 puntos)

Debe completar el proyecto Qt entregado, siguiendo **exactamente** el diseño de la Parte A.

Archivos entregados en AULA USM

- `mainwindow.ui` interfaz espacial lista.
 - `oxygencontroller.h/.cpp` completa.
 - `oxygenhistory.h/.cpp` incompleta (debe completarse).
 - `mainwindow.h/.cpp` incompletos.
 - `CMakeLists.txt` listo.
-

B.1 Implementación de `OxygenHistory` (30 pts)

Debe implementar en `oxygenhistory.h/.cpp`:

- Métodos diseñados en la Parte A. (15 pts)
- `std::vector<float>` manejado correctamente con máximo 12 valores. (5 pts)
- Signals definidos para notificar cambios. (5 pts)
- Slot que reciba % de oxígeno desde `OxygenController`. (5 pts)

Para recibir el puntaje completo, la implementación debe coincidir con lo que escribió en papel.

B.2 Integración con la interfaz (QChart + signals/slots) (30 pts)

Debe completar `MainWindow` agregando:

1. Conexiones faltantes (10 pts)

- `OxygenController` → `OxygenHistory`
- `OxygenHistory` → `MainWindow` (para el gráfico)

2. Implementación de `updateChart()` (15 pts)

- Limpiar la serie del gráfico.
- Recorrer el historial solo mediante métodos públicos de `OxygenHistory`.
- Agregar puntos (`índice`, `valor`) a la serie.
- No acceder directamente al vector.

3. Proyecto funcional (5 pts)

- Compila sin errores.
- Sin crashes.
- El gráfico se actualiza al ajustar los diales.

Para recibir el puntaje completo, la implementación debe coincidir con lo que escribió en papel.

Forma de entrega

Subir un archivo comprimido (`.zip`, `.rar`, `.tar.gz`, etc) que incluya:

- Código fuente (`.h`, `.cpp`, `.ui`).
- `CMakeLists.txt`.
- `Readme.md` con uso de IA (si corresponde).

El proyecto debe compilar en el entorno del curso.

Rúbrica Control 4 (0-100 pts)

Parte A (40 pts)

- **20 pts** Métodos correctos (agregar, contar, acceder) y límite de 12 valores.
- **10 pts** Signals adecuados y coherentes con el diseño.
- **10 pts** Slot correcto para recibir % de oxígeno y conexión razonada.

Parte B (60 pts)

- **30 pts** Implementación de `OxygenHistory` consistente con el diseño (0pts si la entrega de parte A es insuficiente para evaluar).
- **10 pts** Conexiones funcionales (`OxygenController` → `OxygenHistory` → `MainWindow`).
- **15 pts** `updateChart()` funcional usando solo la interfaz pública.
- **5 pts** Proyecto compila y funciona correctamente.